

Introduction to jQuery

Simplifying JavaScript Development

What is jQuery?

- A fast, small, and feature-rich JavaScript library.
- Simplifies tasks like HTML manipulation, event handling, animations, and AJAX.
- Created by John Resig in 2006.

JavaScript libraries



Why Use jQuery?

- 1. Simplifies complex JavaScript tasks.
- 2. Works across major browsers.
- 3. Provides animations and AJAX support.
- 4. Extensible with plugins.
- 5. Strong community support.

Example 1: Changing Text

Vanilla JavaScript:

```
let element = document.getElementById("text");
document.getElementById("btn").addEventListener("click", f
  element.textContent = "Text updated!";
});
```

jQuery:

```
$("#btn").click(function() {
  $("#text").text("Text updated!");
});
```

Example

```
for(i=0; i<document.querySelectorAll("button").length; i++){  
    document.querySelectorAll("button")[i].addEventListener(  
        "click", function(){  
            document.querySelector("h1").style.color="Red";  
        });  
}
```

jQuery



```
$("#button").click(function(){  
    $("#h1").css("color", "red");  
})
```

Including jQuery

- Option 1: Using a CDN
- `<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>`
- Option 2: Download Locally
- 1. Download from jquery.com.
- 2. Include in your project:
- `<script src="path/to/jquery.min.js"></script>`

JavaScript Minifier

- **Definition:**
 - A minifier is a tool that reduces the size of web files (JavaScript, CSS, HTML) by removing unnecessary characters like whitespace, comments, and redundant code.
- **Purpose:**
 - Enhances website speed and performance.
 - Saves bandwidth and reduces loading times.

Example:

Before Minification:

javascript

 Copy code

```
function greet(name) {  
  console.log("Hello, " + name + "!");  
}
```

After Minification:

javascript

 Copy code

```
function greet(n){console.log("Hello, "+n+"!")}
```



Benefits of Using Minifiers

- **Why Use Minifiers?**
 - **Improves Website Performance:**
Smaller file sizes load faster.
 - **Saves Bandwidth:**
Reduced file sizes consume less data.
 - **SEO Advantages:**
Faster loading times improve search engine rankings.
 - **Cost-Effective:**
Less bandwidth means reduced hosting costs.

Minifier.org

- **Introduction:**

- Minifier.org is a free online platform for minifying JavaScript, CSS, and HTML files.
- **How It Works:** Paste or upload your code into the input box.
 - Click the "**Minify**" button.
 - Download or copy the optimized code.

Selecting Element with jQuery

- **Definition:** jQuery selectors allow developers to find and manipulate HTML elements efficiently.
- **Purpose:**
 - Traverse and interact with the DOM.
 - Apply styles, retrieve content, and attach events.

Syntax and Example

- **Basic Syntax:**

javascript

 Copy code

```
$("#selector").action();
```

- **Example:**

javascript

 Copy code

```
$("#p").css("color", "blue");
```

Common jQuery Selectors

- Common Types of Selectors
- **1. Basic Selectors:**
 - Element: `$("p")` (Selects all `<p>` elements).
 - ID: `$("#myId")` (Selects element with `id="myId"`).
 - Class: `$(".myClass")` (Selects elements with `class="myClass"`).

Common jQuery Selectors

- Specific Attribute:

javascript

 Copy code

```
$("#input[name='email']");
```

- Attribute Contains:

javascript

 Copy code

```
$("#a[href*='example']");
```

Common jQuery Selectors

- **3. Hierarchical Selectors:**
 - descendant: `$("div p")` (Selects `<p>` inside `<div>`)
 - Child: `$("ul > li")` (Direct `<li>` children of `<ul>`).

Manipulating Styles with jQuery

- **Definition:**

jQuery makes it easy to dynamically modify the styles of HTML elements using built-in methods.

- **Purpose:**

- Enhance interactivity.
- Dynamically update the user interface

Manipulating Styles with jQuery

- **Common Methods:**
 - `.css()` – Modify or retrieve styles.
 - `.addClass()` – Add a CSS class.
 - `.removeClass()` – Remove a CSS class.
 - `.toggleClass()` – Toggle a CSS class.

Examples of Style Manipulation

1. Change CSS Properties:

javascript

 Copy code

```
$("#p").css("color", "blue");
```

Changes all paragraphs to blue.

2. Add/Remove Classes:

javascript

 Copy code

```
$("#div").addClass("highlight");  
$("#div").removeClass("hi  
ight");
```

Examples of Style Manipulation

3. Toggle a Class:

javascript

 Copy code

```
$("#box").toggleClass("active");
```

4. Retrieve a Style:

javascript

 Copy code

```
let color = $("p").css("color");  
console.log(color);
```

Text Manipulation in jQuery

- **Definition:**
jQuery allows you to dynamically get or set the text content of HTML elements.
- **Purpose:**
 - Update user-visible content dynamically.
 - Retrieve and modify text easily without reloading the page.
- **Common Methods:**
 - `.text()` – Get or set the text content.
 - `.html()` – Get or set the HTML content.
 - `.val()` – Get or set the value of form elements.

Examples of Text Manipulation

1. Get Text Content:

javascript

 Copy code

```
let text = $("p").text();  
console.log(text);
```

2. Set Text Content:

javascript

 Copy code

```
$("p").text("This is new text.");
```



Examples of Text Manipulation

3. Get HTML Content:

javascript

 Copy code

```
let htmlContent = $("#content").html();  
console.log(htmlContent);
```

4. Set HTML Content:

javascript

 Copy code

```
$("#content").html("<strong>Bold Text</strong>");
```

Examples of Text Manipulation

5. Get or Set Form Input Values:

javascript

 Copy code

```
let inputValue = $("#name").val(); // Get value
$("#name").val("New Value"); // Set value
```


Attribute Manipulation in jQuery

- **Definition:**
jQuery provides methods to get, set, or remove attributes from HTML elements dynamically.
- **Purpose:**
 - Dynamically update element properties like src, href, alt, and more.
 - Enhance interactivity and improve user experience.
- **Common Methods:**
 - .attr() – Get or set an attribute.
 - .removeAttr() – Remove an attribute.
 - .prop() – Get or set a property (e.g., checked, disabled).

Common Methods for Attribute Manipulation

1. Get an Attribute Value:

javascript

 Copy code

```
let href = $("a").attr("href");  
console.log(href); // Outputs the href value of the first <a>
```

2. Set an Attribute Value:

javascript

 Copy code

```
$("img").attr("src", "new-image.jpg");
```

Common Methods for Attribute Manipulation

3. Remove an Attribute:

javascript

 Copy code

```
$("#input").removeAttr("disabled");
```

4. Toggle a Property:

javascript

 Copy code

```
$("#checkbox").prop("checked", true); // Check the checkbox
```

What Are Event Listeners in jQuery?

- **Definition:**
Event listeners in jQuery handle user interactions (e.g., clicks, mouse movements, keypresses) efficiently.
- **Purpose:**
 - Respond to user actions dynamically.
 - Enhance the interactivity of web applications.
- **Common Methods:**
- `.on()` – Add an event listener.
- `.off()` – Remove an event listener.
- **Shorthand Methods:**
 - `.click()`
 - `.hover()`
 - `.keydown()`

Common Event Listener Methods

- 1. Using .on() to Bind an Event:

```
$("#btn").on("click", function () {  
    alert("Button clicked!");  
});
```

- 2.Using Shorthand Methods:
 - Click Event:

```
$("#button").click(function () {  
    alert("Button was clicked!");  
});
```

Common Event Listener Methods

– Hover Event:

```
$(".box").hover(  
    function () {  
        $(this).css("background-color", "yellow");  
    },  
    function () {  
        $(this).css("background-color", "");  
    }  
);
```

• Removing Event Listeners:

```
$("#btn").off("click");
```

Adding Elements with jQuery

- Methods to Add Elements
 - **append()**: Adds content to the end of an element.

```
$("#container").append("<p>New Paragraph</p>");
```

- **prepend()**: Adds content to the beginning.

```
$("#container").prepend("<p>Start Here</p>");
```

- **after()**: Inserts content after an element.

```
$("#element").after("<p>Next to it!</p>");
```

- **before()**: Inserts content before an element.

```
$("#element").before("<p>Above it!</p>");
```

Removing Elements

- Methods to Remove Elements
 - **remove()**: Deletes an element and its children.

```
$("#element").remove();
```

- **empty()**: Clears the content of an element.

```
$("#container").empty();
```


Website Animations with jQuery

- **Why Use jQuery for Animations?**
 - Simplifies animations with built-in methods.
 - No need for complex CSS or JavaScript.
- **Key Animation Methods:**
 - `show()` / `hide()`
 - `fadeIn()` / `fadeOut()`
 - `slideDown()` / `slideUp()`
 - `animate()`

Basic Animations

- Predefined Animation Methods
 - **Show and Hide:**

```
$("#element").show();  
$("#element").hide();
```

```
$("#element").fadeIn(1000); // 1 second  
$("#element").fadeOut(500); // 0.5 seconds
```

Basic Animations

– Fade Effects:

```
$("#element").slideDown();  
$("#element").slideUp();
```

Custom Animations

- Custom Animation with `animate()`
 - **What It Does:** Change CSS properties over time.
 - **Syntax:**

```
$("#element").animate({  
  opacity: 0.5,  
  left: "50px"  
}, 1000); // Duration: 1 second
```

```
$("#element").fadeIn(500).animate({  
  width: "200px"  
}, 1000);
```

Best Practices

- 1. Include the latest stable version.
- 2. Use CDN for faster loading.
- 3. Minimize global variables.
- 4. Use `.on()` for event delegation.
- 5. Prefer modern frameworks for large-scale applications.

Alternatives

- - Vanilla JavaScript: Use modern APIs like `querySelector`.
- - Frameworks: React, Angular, Vue.js.

Conclusion

- - jQuery is still relevant for simple tasks and legacy systems.
- - For modern apps, consider native JavaScript or frameworks.

References

- 1. Official jQuery Website: <https://jquery.com/>
- 2. jQuery Documentation: <https://api.jquery.com/>
- 3. jQuery Plugins: <https://plugins.jquery.com/>